

Cloud Compliance Automation Platform - Project Plan

A single end-to-end AWS compliance automation project, governed through ServiceNow IRM, built to provide real, demonstrable, hands-on GRC engineering experience.

The platform runs in five layers: a Terraform-based infrastructure baseline with policy-as-code CI/CD; a continuously monitored AWS environment; a Python automation and evidence layer; a ServiceNow IRM governance layer with role-based access control; and a formal control testing program tying it all together, reported through ServiceNow Performance Analytics.

All code is written in Python.

Phase 0 - Foundations

Set up the environment everything else will run in.

- Create an AWS sandbox account with billing alarms/budgets configured
 - Create a GitHub repository with branch protection rules
 - Create a ServiceNow Personal Developer Instance (PDI) and activate the IRM plugin
 - Configure Terraform remote state (S3 backend with DynamoDB locking)
-

Phase 1 - Compliant infrastructure baseline (Terraform)

Build the core AWS environment as code, with a couple of realistic, intentional issues seeded in for later phases to detect and remediate.

- Write Terraform modules for VPC, IAM (users/groups/roles designed around RBAC), S3, and EC2/Lambda compute
 - Seed an IAM user with AdministratorAccess and no MFA
 - Seed an S3 bucket without encryption at rest
 - Document the "before" state for later reference
-

Phase 2 - Policy-as-code CI/CD pipeline *(revised)*

Enforce governance at the pipeline level, before infrastructure is ever deployed.

- Build a GitHub Actions workflow: `terraform plan` -> Checkov/tfsec scan (mapped to the CIS AWS Benchmark) -> Bandit SAST scan on the Python code -> dependency vulnerability scan (pip-audit) on Python dependencies
- Configure the pipeline to block `terraform apply` on critical/high findings from any stage
- Document which policy-as-code rules map to which framework controls

- Track any accepted findings (false positives, accepted risk) in a lightweight vulnerability log with owner and justification - this is what turns "ran a scanner" into an actual vulnerability management process

Change log: Removed the optional OWASP ZAP DAST pass (no demo app existed elsewhere in the plan to justify it, and DAST was dropped as a resume keyword). Replaced with pip-audit dependency scanning plus a vulnerability acceptance log, to give the "Vulnerability Management" keyword real substance instead of just running scanners.

Phase 3 - Continuous cloud compliance monitoring

Turn on AWS's native continuous monitoring services and confirm they correctly detect the Phase 1 issues.

- Enable AWS Config with a conformance pack mapped to ISO 27001/NIST CSF
 - Enable Security Hub with the CIS AWS Foundations and AWS FSBP standards
 - Enable GuardDuty
 - Enable an organization CloudTrail
 - Enable Audit Manager using its built-in ISO 27001 and SOC 2 frameworks
 - Confirm the seeded IAM and S3 issues are flagged correctly
-

Phase 4 - IAM governance automation (Python/boto3)

Automate detection and remediation of excessive privileges and dormant accounts.

- Write a scheduled script/Lambda that scans IAM for: users without MFA, access keys older than 90 days, roles unused for 90+ days, and policies granting AdministratorAccess
 - Generate a remediation report from the scan
 - Add an approval-gated remediation step (e.g. deactivating the worst offenders)
 - Run it against the Phase 1 environment and capture before/after evidence
-

Phase 5 - Automated evidence collection pipeline

Automate the collection and organisation of audit evidence.

- Write boto3 scripts/Lambdas to pull Config compliance snapshots, Security Hub findings, CloudTrail samples, and Audit Manager reports
 - Store outputs as timestamped artifacts in S3, organised by control ID
 - Build an initial control matrix mapping each evidence artifact to ISO 27001 Annex A clauses and SOC 2 Trust Services Criteria
-

Phase 6 - Control testing program

Apply a formal, industry-standard control testing methodology to the controls built so far.

- Expand the control matrix into a full Risk & Control Matrix (RCM): control ID, description, framework mappings, owner, control type (preventive/detective/corrective), nature (automated/manual/hybrid), and frequency
 - For controls where a script has full API access to the population (e.g. all IAM users, all S3 buckets), test 100% of the population
 - For manual/periodic controls, define a population and select an appropriate sample
 - For automated controls, test by **re-performance**: reintroduce a seeded issue and time how long the control takes to detect it
 - For manual controls, test by **inspection**: check sampled evidence artifacts against criteria
 - Document results in a workpaper: Control ID | Test date | Procedure | Sample/population | Result | Evidence reference | Notes
 - Log any exception as a ServiceNow Issue with an owner, remediation plan, and retest date
-

Phase 7 - ServiceNow IRM integration

Build out the ServiceNow governance layer and connect it to the AWS environment.

- Configure the Policy & Compliance module: Authority Documents for ISO 27001, SOC 2, and NIST CSF, with linked Control Objectives
 - Set up the Risk Register
 - Define GRC roles: control owner, risk manager, auditor (read-only), admin
 - Configure ServiceNow ACL rules on the IRM tables (Controls, Risk Register, Issues, Compliance Assessments, Authority Documents) to enforce least privilege and segregation of duties
 - e.g. the person who raises/remediates an Issue cannot be the one who closes it
 - Build a Python integration against the ServiceNow Table API that: pushes Security Hub findings as Issues, updates Compliance Assessment results from AWS Config's compliance state, and attaches evidence links from the S3 evidence store
 - Add the ACL/segregation-of-duties configuration as a control in the RCM, and test it by re-performance - create test users for each role, attempt restricted actions, confirm correct allow/deny
-

Phase 8 - GRC reporting

Produce the dashboards a GRC stakeholder would actually look at.

- Build a ServiceNow Performance Analytics dashboard: control effectiveness rate, risk heatmap, issue ageing, audit readiness score

- Optionally, build a supporting Grafana dashboard from CloudWatch metrics: GuardDuty alert volume, CloudTrail activity trends, Config compliance drift
-

Phase 9 - Governance documentation *(revised)*

Produce the supporting governance artifacts that round out a GRC program.

- Finalise the Risk Register with entries derived from GuardDuty/Security Hub findings, each scored using a likelihood x impact methodology (e.g. 1-5 scale on each axis) to produce an inherent and residual risk rating per entry
 - Write a gap analysis comparing the platform's controls against ISO 27001, SOC 2, NIST CSF, and CIS Benchmarks
 - Write a lightweight TPRM assessment treating ServiceNow (as a SaaS dependency) as a third party

Change log: Added an explicit likelihood x impact risk scoring methodology to the Risk Register bullet, to give "Risk Assessments" real backing rather than just a list of findings. Removed the data flow review (personal data/GDPR) bullet, since GDPR was dropped as a resume keyword.

Phase 10 - Defence pack and demo

Package the work into something that can be walked through end to end.

- Write an architecture diagram and README for the repository
 - Record a short demo walkthrough
 - Prepare a short narrative for each phase: what was built, why, and what it demonstrates
-

Resume bullet -> phase mapping

Resume bullet	Phase(s)
Terraform baseline + Checkov/tfsec + Bandit gating via GitHub Actions	1, 2
Continuous compliance monitoring (Config, Security Hub, GuardDuty, CloudTrail, Audit Manager)	3
Python tool for ISO 27001/SOC 2/NIST CSF + evidence collection	4, 5
Formal control testing program (RCM, sampling, inspection/re-performance, exceptions)	6
ServiceNow IRM integration (REST API, Authority Documents, Risk Register, Compliance Assessments, ALPS, etc)	7
Assessments and Performance Analytics	8
Vulnerability Management (keyword)	2
Risk Assessments (keyword)	9
Gap Analysis, TPRM (keywords)	9